



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA



ACCREDITED GRADE 'A'
BY NAAC



PERFECT SCORE OF **150/150** AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS

1



Applied Machine Learning

Unit 05 – Lecture 12: Regularization – Ridge and Lasso

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Ridge, lasso, elastic net, and how to choose the penalty

Topic focus: **deliberately fitting the training data worse, on purpose**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.; Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.; Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.

Why
○○○○

Ridge
○○○○○○○○○○○○○○○○

Lasso
○○○○○○○○○

Paths
○○○

Scaling
○○○○○

Choosing λ
○○

Bias–variance
○○○

Elastic net
○○

Summary
○○○

Quick Links

Why

Ridge

Lasso

Paths

Scaling

Choosing λ

Bias–variance

Elastic net

Summary

Agenda

The problem regularization solves

Ridge regression

Lasso

Coefficient paths

Scaling is not optional

Choosing λ by cross-validation

The trade being made

Elastic net

Summary

How this lecture works

Every idea appears twice

1. **By hand** — three numbers, a 2×2 inverse, four OLS coefficients. Small enough to check on paper.
2. **In code** — the same arithmetic on real data, so you see that Ridge and Lasso do exactly that and nothing more.

Commit before you look

Four slides ask you to predict a number before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

The centrepiece is **one figure**: ten coefficient curves under each penalty. Ridge decays them smoothly and never reaches zero; lasso kills them one at a time.

One dataset, one seed

The data

scikit-learn's bundled `load_diabetes`, in both its pre-scaled and its original-units form. Everything else is synthetic, drawn from the same seed. Nothing is downloaded.

The randomness

One seed for the whole lecture:
`np.random.default_rng(0)` and `random_state=0`. Every listing that draws, splits or resamples shows it. Where the *spread* is the point — twenty $p > n$ draws, thirty bootstrap resamples — the seeds are $0, 1, 2, \dots$, derived from that one.

Why this matters to you

Every number in this deck is printed by one script. Run the code as written and you get the same numbers — which is the only reason to believe a number on a slide.

Where we are: three failures, one cure

Unit V raised	What goes wrong	What the penalty does
Over-fitting	the fitted surface chases the noise; huge $\ \beta\ $	caps $\ \beta\ $, so the surface cannot wiggle
Multicollinearity	$X^T X$ nearly singular; coefficients huge, of opposite sign, unstable	adds λ to the diagonal, so the matrix is well conditioned again
Perfect separation	the logistic likelihood has no maximum; $\ \beta\ \rightarrow \infty$	the penalty is the only thing stopping it

Stop asking for the smallest training error

$$\hat{\beta} = \arg \min_{\beta} \underbrace{\text{loss}(\beta)}_{\text{fit the data}} + \lambda \underbrace{\text{pen}(\beta)}_{\text{stay small}}$$

$\lambda = 0$ gives back the unpenalised fit; $\lambda \rightarrow \infty$ gives the constant model. Everything interesting is in between — and λ is **not fitted**, it is chosen by cross-validation, which is why this lecture could not come before the validation material.

Predict first #1

$n = 40$ rows, $p = 38$ columns of pure noise, and only *three* of them carry any signal at all. Fit ordinary least squares. Then fit ridge with $\alpha = 10$.

What does each score on 2000 fresh rows?

Predict first #1

$n = 40$ rows, $p = 38$ columns of pure noise, and only *three* of them carry any signal at all. Fit ordinary least squares. Then fit ridge with $\alpha = 10$.

What does each score on 2000 fresh rows?

(a) over-fitting	n=40	p=38				
OLS	train R2	+0.9997	test R2	+0.1470	beta	5.075
ridge	train R2	+0.9336	test R2	+0.6598	beta	2.432

Predict first #1

$n = 40$ rows, $p = 38$ columns of pure noise, and only *three* of them carry any signal at all. Fit ordinary least squares. Then fit ridge with $\alpha = 10$.

What does each score on 2000 fresh rows?

(a)	over-fitting	n=40	p=38			
	OLS	train R^2	+0.9997	test R^2	+0.1470	$ \beta $ 5.075
	ridge	train R^2	+0.9336	test R^2	+0.6598	$ \beta $ 2.432

Ridge fits the training data *worse*, on purpose

Training R^2 falls from 0.9997 to 0.9336 — and test R^2 rises from 0.1470 to **0.6598**, a factor of four and a half. The coefficient vector halves in length. **Every point of training fit given up bought several points of test fit.**

Multicollinearity: the coefficients will not stand still

Two columns with $\text{corr} = 0.99989$; the truth is $y = 2z$.

```
(b) multicollinearity corr(x1,x2) = 0.999885
```

```
OLS coefficients [ 6.5628 -4.54 ]
```

```
ridge coefficients [1.0549 0.9486]
```

```
the same fit on five bootstrap resamples of the same 100 rows:
```

OLS	[+11.0217, -8.9539]	sum +2.0677	ridge	[+1.1020, +0.9276]	sum +2.0297
OLS	[+2.7446, -0.6954]	sum +2.0492	ridge	[+1.0332, +0.9985]	sum +2.0317
OLS	[+6.7951, -4.7408]	sum +2.0543	ridge	[+1.0785, +0.9529]	sum +2.0315
OLS	[+5.0355, -3.0251]	sum +2.0105	ridge	[+1.0354, +0.9583]	sum +1.9937
OLS	[+7.5289, -5.5234]	sum +2.0055	ridge	[+1.0785, +0.9294]	sum +1.9867

Multicollinearity: the coefficients will not stand still

Two columns with $\text{corr} = 0.99989$; the truth is $y = 2z$.

```
(b) multicollinearity corr(x1,x2) = 0.999885
```

```
OLS coefficients [ 6.5628 -4.54 ]
```

```
ridge coefficients [1.0549 0.9486]
```

```
the same fit on five bootstrap resamples of the same 100 rows:
```

OLS [+11.0217, -8.9539]	sum +2.0677	ridge [+1.1020, +0.9276]	sum +2.0297
OLS [+2.7446, -0.6954]	sum +2.0492	ridge [+1.0332, +0.9985]	sum +2.0317
OLS [+6.7951, -4.7408]	sum +2.0543	ridge [+1.0785, +0.9529]	sum +2.0315
OLS [+5.0355, -3.0251]	sum +2.0105	ridge [+1.0354, +0.9583]	sum +1.9937
OLS [+7.5289, -5.5234]	sum +2.0055	ridge [+1.0785, +0.9294]	sum +1.9867

Look down the two “sum” columns, then at the pairs

The *sum* is stable in both — always about 2.0, because that is what the data determines. The *split* is not: OLS puts +11.0 on one column and −9.0 on the other in the first resample and +2.7/ − 0.7 in the second. **Ridge splits it evenly every time.**

And the L10 case: perfect separation

Six points, one feature, the classes perfectly separated. The maximum likelihood estimate does not exist — the slope runs away.

(c) perfect separation, 6 points, one feature

$C = 1e+10$	penalty strength $1/C = 1e-10$	fitted slope	8.3469
$C = 1$	penalty strength $1/C = 1$	fitted slope	1.1043
$C = 0.01$	penalty strength $1/C = 100$	fitted slope	0.0561

And the L10 case: perfect separation

Six points, one feature, the classes perfectly separated. The maximum likelihood estimate does not exist — the slope runs away.

(c) perfect separation, 6 points, one feature

$C = 1e+10$	penalty strength $1/C = 1e-10$	fitted slope	8.3469
$C = 1$	penalty strength $1/C = 1$	fitted slope	1.1043
$C = 0.01$	penalty strength $1/C = 100$	fitted slope	0.0561

LogisticRegression in scikit-learn is **L2-penalised by default**, with $C = 1$, and C is $1/\lambda$.
You have been using regularization since L10 without being told.

And the L10 case: perfect separation

Six points, one feature, the classes perfectly separated. The maximum likelihood estimate does not exist — the slope runs away.

(c) perfect separation, 6 points, one feature

$C = 1e+10$	penalty strength $1/C = 1e-10$	fitted slope	8.3469
$C = 1$	penalty strength $1/C = 1$	fitted slope	1.1043
$C = 0.01$	penalty strength $1/C = 100$	fitted slope	0.0561

LogisticRegression in scikit-learn is **L2-penalised by default**, with $C = 1$, and C is $1/\lambda$.
You have been using regularization since L10 without being told.

Turn the penalty off ($C = 10^{10}$) and the slope climbs to 8.35 and would keep climbing with more iterations. **The penalty is what makes the answer exist.**

Ridge: the objective, two ways of writing it

Penalised form

$$J(\beta) = \underbrace{(y - X\beta)^T (y - X\beta)}_{\text{RSS}} + \lambda \beta^T \beta, \quad \lambda \geq 0.$$

Constrained form — the same problem

$$\min_{\beta} (y - X\beta)^T (y - X\beta) \quad \text{subject to} \quad \|\beta\|_2^2 \leq s.$$

Two things to fix in your head now

The intercept is not penalised. β_0 is excluded from the sum; otherwise the model could not even represent a constant shift in y .

λ is not a parameter. It is a hyper-parameter: it is not estimated from the data by the fit, it is chosen from outside by cross-validation.

Deriving the closed form — four lines

Differentiate. $\frac{\partial}{\partial \beta} (y - X\beta)^\top (y - X\beta) = -2X^\top (y - X\beta)$ and $\frac{\partial}{\partial \beta} \lambda \beta^\top \beta = 2\lambda \beta$.

$$\nabla J(\beta) = -2X^\top (y - X\beta) + 2\lambda \beta = 0$$

$$\implies X^\top y = X^\top X \beta + \lambda \beta = (X^\top X + \lambda I) \beta$$

$$\implies \boxed{\hat{\beta}^{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y}$$

Set $\lambda = 0$ and you get least squares back

$\hat{\beta}^{\text{OLS}} = (X^\top X)^{-1} X^\top y$, the L8 formula. Ridge is one extra term on the diagonal — and that one term is the whole lecture.

The Hessian is $2(X^\top X + \lambda I)$, which we are about to show is positive definite, so the stationary point is the unique minimum.

The structural point: the inverse always exists

Claim

For every $\lambda > 0$, the matrix $X^T X + \lambda I$ is positive definite, and therefore invertible — **whatever X is.**

Proof, one line

For any $v \neq 0$,

$$v^T (X^T X + \lambda I) v = \|Xv\|^2 + \lambda \|v\|^2 \geq \lambda \|v\|^2 > 0.$$

The first term can be zero — that is exactly what a singular $X^T X$ means — but the second cannot.

The same thing in eigenvalues

If $X^T X$ has eigenvalues $d_1^2 \geq \dots \geq d_p^2 \geq 0$, then $X^T X + \lambda I$ has eigenvalues $d_j^2 + \lambda$, all strictly positive. **Adding λ to the diagonal lifts every eigenvalue off the floor.**

So ridge has an answer in cases where *ordinary least squares does not exist at all*. That is not a refinement; it is a change of kind.

By hand #1: three numbers, one feature

$x = (1, 2, 2)$, $y = (2, 3, 5)$, no intercept. Then $X^T X = 1 + 4 + 4 = 9$ and $X^T y = 2 + 6 + 10 = 18$, so

$$\hat{\beta}(\lambda) = \frac{18}{9 + \lambda} = \underbrace{2}_{\beta^{\text{OLS}}} \times \underbrace{\frac{9}{9 + \lambda}}_{\text{shrinkage factor}}.$$

```
for lam in (0., 1., 9., 81.):
    print(Ridge(alpha=lam, fit_intercept=False).fit(x.reshape(-1,1), y).coef_)
```

lambda	0	beta = 18 / (9 + 0) = 2.0000	shrinkage factor 1.0000	sklearn 2.0000
lambda	1	beta = 18 / (9 + 1) = 1.8000	shrinkage factor 0.9000	sklearn 1.8000
lambda	9	beta = 18 / (9 + 9) = 1.0000	shrinkage factor 0.5000	sklearn 1.0000
lambda	81	beta = 18 / (9 + 81) = 0.2000	shrinkage factor 0.1000	sklearn 0.2000

By hand #1: three numbers, one feature

$x = (1, 2, 2)$, $y = (2, 3, 5)$, no intercept. Then $X^T X = 1 + 4 + 4 = 9$ and $X^T y = 2 + 6 + 10 = 18$, so

$$\hat{\beta}(\lambda) = \frac{18}{9 + \lambda} = \underbrace{2}_{\beta_{\text{OLS}}} \times \underbrace{\frac{9}{9 + \lambda}}_{\text{shrinkage factor}}.$$

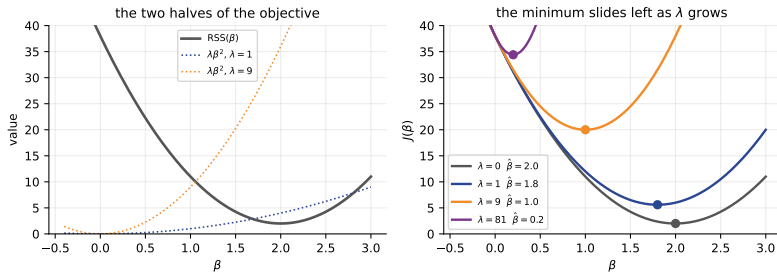
```
for lam in (0., 1., 9., 81.):
    print(Ridge(alpha=lam, fit_intercept=False).fit(x.reshape(-1,1), y).coef_)
```

lambda	0	beta = 18 / (9 + 0) = 2.0000	shrinkage factor 1.0000	sklearn 2.0000
lambda	1	beta = 18 / (9 + 1) = 1.8000	shrinkage factor 0.9000	sklearn 1.8000
lambda	9	beta = 18 / (9 + 9) = 1.0000	shrinkage factor 0.5000	sklearn 1.0000
lambda	81	beta = 18 / (9 + 81) = 0.2000	shrinkage factor 0.1000	sklearn 0.2000

At $\lambda = X^T X$ the coefficient is exactly halved

Four digits, four matches: Ridge is $(X^T X + \lambda I)^{-1} X^T y$ and nothing else. And note that λ is measured **on the same scale as $X^T X$** — whether 9 is “a lot” depends entirely on the size of the numbers in X , which is why Section 5 exists.

Drawn: the minimum slides towards the origin



Left: RSS is a parabola with its minimum at 2; $\lambda\beta^2$ is a parabola with its minimum at 0. Right: their sum, for four values of λ . **Adding a bowl centred at the origin drags the minimum towards the origin, and never past it.**

By hand #2: a 2×2 inverse

$X = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}$, $y = (1, 2, 3)^T$. Then $X^T X = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ and $X^T y = (4, 5)^T$.

λ	$X^T X + \lambda I$	det	$\hat{\beta}$	$\ \hat{\beta}\ _2$
0	$\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$	3	(1.0000, 2.0000)	2.2361
1	$\begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$	8	(0.8750, 1.3750)	1.6298
3	$\begin{pmatrix} 5 & 1 \\ 1 & 5 \end{pmatrix}$	24	(0.6250, 0.8750)	1.0753

Check the $\lambda = 1$ row by hand: $\frac{1}{8} \begin{pmatrix} 3 & -1 \\ -1 & 3 \end{pmatrix} \begin{pmatrix} 4 \\ 5 \end{pmatrix} = \frac{1}{8} \begin{pmatrix} 7 \\ 11 \end{pmatrix} = (0.875, 1.375)$.

Notice they do not shrink at the same rate

β_1 falls 12.5% then a further 29%; β_2 falls 31% then a further 36%. **Ridge is not a uniform scaling of the OLS vector** unless $X^T X$ is a multiple of the identity.

Predict first #2

Now make the second column **exactly twice** the first: $X = \begin{pmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{pmatrix}$, so $X^T X = \begin{pmatrix} 14 & 28 \\ 28 & 56 \end{pmatrix}$.

What does `np.linalg.inv(X.T @ X)` return? And what does `LinearRegression` return?

Predict first #2

Now make the second column **exactly twice** the first: $X = \begin{pmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{pmatrix}$, so $X^T X = \begin{pmatrix} 14 & 28 \\ 28 & 56 \end{pmatrix}$.

What does `np.linalg.inv(X.T @ X)` return? And what does `LinearRegression` return?

```
det(X'X) = 0.0      rank(X'X) = 1      eigenvalues [ 0. 70.]  
np.linalg.inv(X'X) raises LinAlgError: Singular matrix  
np.linalg.solve(X'X, X'y) raises: LinAlgError: Singular matrix  
what LinearRegression does instead: lstsq gives the MINIMUM-NORM  
solution, not 'the' solution: [0.2 0.4]
```


Predict first #2

Now make the second column **exactly twice** the first: $X = \begin{pmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{pmatrix}$, so $X^T X = \begin{pmatrix} 14 & 28 \\ 28 & 56 \end{pmatrix}$.

What does `np.linalg.inv(X.T @ X)` return? And what does `LinearRegression` return?

```
det(X'X) = 0.0      rank(X'X) = 1      eigenvalues [ 0. 70.]
np.linalg.inv(X'X) raises LinAlgError: Singular matrix
np.linalg.solve(X'X, X'y) raises: LinAlgError: Singular matrix
what LinearRegression does instead: lstsq gives the MINIMUM-NORM
solution, not 'the' solution: [0.2 0.4]
```

The formula raises. The library does not.

`LinearRegression` calls `lstsq`, which when the answer is not unique quietly returns **the solution of smallest $\|\beta\|_2$** . That is a choice, and nothing in the output tells you it was made. **You will meet this every time $p > n$.**

Ridge, on the same singular problem

```
lambda 1  X'X + 1 I =[[15 28] [28 57]]  det  71.0  eigenvalues [ 1. 71.]
           beta = [0.19718, 0.39437]    beta2 / beta1 = 2.0000
lambda 4  X'X + 4 I =[[18 28] [28 60]]  det 296.0  eigenvalues [ 4. 74.]
           beta = [0.18919, 0.37838]    beta2 / beta1 = 2.0000
sklearn Ridge(alpha=1, fit_intercept=False).coef_ = [0.19718 0.39437]
```

Ridge, on the same singular problem

```
lambda 1  X'X + 1 I =[[15 28] [28 57]]  det  71.0  eigenvalues [ 1. 71.]
           beta = [0.19718, 0.39437]    beta2 / beta1 = 2.0000
lambda 4  X'X + 4 I =[[18 28] [28 60]]  det 296.0  eigenvalues [ 4. 74.]
           beta = [0.18919, 0.37838]    beta2 / beta1 = 2.0000
sklearn Ridge(alpha=1, fit_intercept=False).coef_ = [0.19718 0.39437]
```

The determinant went from **0** to **71** by adding 1 to two diagonal entries. The eigenvalue that was 0 is now λ . **The problem was not hard; it was ill-posed, and ridge posed it properly.**

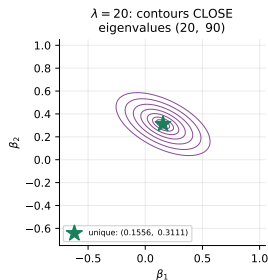
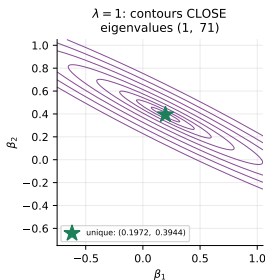
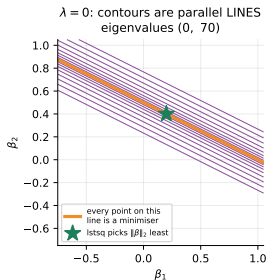
Ridge, on the same singular problem

```
lambda 1  X'X + 1 I =[[15 28] [28 57]]  det  71.0  eigenvalues [ 1. 71.]
           beta = [0.19718, 0.39437]    beta2 / beta1 = 2.0000
lambda 4   X'X + 4 I =[[18 28] [28 60]]  det 296.0  eigenvalues [ 4. 74.]
           beta = [0.18919, 0.37838]    beta2 / beta1 = 2.0000
sklearn Ridge(alpha=1, fit_intercept=False).coef_ = [0.19718 0.39437]
```

The determinant went from **0** to **71** by adding 1 to two diagonal entries. The eigenvalue that was 0 is now λ . **The problem was not hard; it was ill-posed, and ridge posed it properly.**

And read $\beta_2/\beta_1 = 2.0000$: ridge splits the weight between two identical directions *in proportion to their column norms*, rather than picking one. That is the grouping behaviour we come back to at the end.

Drawn: a valley, then a bowl



Left: with a singular $X^T X$ the RSS contours are *parallel straight lines* — every point on the orange line fits the three rows perfectly, so there is no unique answer. Middle and right: adding λI closes the contours into ellipses, and the smallest eigenvalue is exactly λ .

$p > n$: the case OLS cannot even state

$n = 50$ rows, $p = 200$ columns, five of which carry signal.

```
rank(X'X) = 50 of 200 -- X'X is singular by construction
```

```
number of zero eigenvalues of X'X: 150
```

```
smallest eigenvalue of X'X      : -1.812e-13
```

```
smallest eigenvalue of X'X + 10I : 1.000e+01
```

OLS (lstsq min-norm)	train R2	+1.0000	test R2	+0.2831	beta	3.178	nonzero	200
----------------------	----------	---------	---------	---------	------	-------	---------	-----

Ridge, CV alpha 0.1	train R2	+1.0000	test R2	+0.2831	beta	3.176	nonzero	200
---------------------	----------	---------	---------	---------	------	-------	---------	-----

Lasso, CV alpha 0.215	train R2	+0.9807	test R2	+0.9294	beta	5.268	nonzero	20
-----------------------	----------	---------	---------	---------	------	-------	---------	----

$p > n$: the case OLS cannot even state

$n = 50$ rows, $p = 200$ columns, five of which carry signal.

```
rank(X'X) = 50 of 200 -- X'X is singular by construction
```

```
number of zero eigenvalues of X'X: 150
```

```
smallest eigenvalue of X'X      : -1.812e-13
```

```
smallest eigenvalue of X'X + 10I : 1.000e+01
```

```
  OLS (lstsq min-norm)   train R2 +1.0000   test R2 +0.2831   ||beta||    3.178   nonzero 200
```

```
  Ridge, CV alpha 0.1    train R2 +1.0000   test R2 +0.2831   ||beta||    3.176   nonzero 200
```

```
  Lasso, CV alpha 0.215  train R2 +0.9807   test R2 +0.9294   ||beta||    5.268   nonzero  20
```

```
what lstsq actually returned: the MINIMUM-NORM solution, which is
```

```
exactly the limit of ridge as lambda -> 0:
```

```
  alpha 1e-09   max |ridge - lstsq| 6.487e-12   ||ridge|| 3.1907   ||lstsq|| 3.1907
```

$p > n$: the case OLS cannot even state

$n = 50$ rows, $p = 200$ columns, five of which carry signal.

```
rank(X'X) = 50 of 200 -- X'X is singular by construction
```

```
number of zero eigenvalues of X'X: 150
```

```
smallest eigenvalue of X'X      : -1.812e-13
```

```
smallest eigenvalue of X'X + 10I : 1.000e+01
```

```
  OLS (lstsq min-norm)   train R2 +1.0000   test R2 +0.2831   ||beta||    3.178   nonzero 200
```

```
  Ridge, CV alpha 0.1    train R2 +1.0000   test R2 +0.2831   ||beta||    3.176   nonzero 200
```

```
  Lasso, CV alpha 0.215  train R2 +0.9807   test R2 +0.9294   ||beta||    5.268   nonzero  20
```

```
what lstsq actually returned: the MINIMUM-NORM solution, which is
```

```
exactly the limit of ridge as lambda -> 0:
```

```
alpha 1e-09   max |ridge - lstsq| 6.487e-12   ||ridge|| 3.1907   ||lstsq|| 3.1907
```

Two honest readings

One: “OLS” at $p > n$ is already a regularised estimator — it is ridge at $\lambda \rightarrow 0$, agreeing to 6×10^{-12} .

Two: on a *sparse* truth ridge does nothing at all — CV picks the smallest α on the grid and hands back the OLS fit (0.2831) — while lasso is transformative (**0.9294**), recovering all five real columns. Ridge is not always the answer.

When ridge earns its keep: $p > n$ and correlated

Same shape, but the 100 columns are driven by five latent factors and **all** 100 coefficients are non-zero. One draw, then twenty.

```
n = 50, p = 100, columns driven by 5 latent factors (rho = 0.9),
mean |correlation| between distinct columns: 0.3370
  OLS (lstsq min-norm)      train R2 +1.0000  test R2 +0.4183  nonzero 100
  Ridge, CV alpha 5.2      train R2 +0.9469  test R2 +0.6303  nonzero 100
  Lasso, CV alpha 0.152    train R2 +0.8298  test R2 +0.6618  nonzero 14
one draw settles nothing at n = 50. repeating over 20 draws,
seeds SEED..SEED+19:
  mean test R2   OLS +0.4446   ridge +0.6532   lasso +0.6443
  worst OLS draw -0.4915; OLS is worse than predicting the mean in 1 of 20
  ridge beats OLS in 19 of 20 draws, lasso in 16 of 20
```

When ridge earns its keep: $p > n$ and correlated

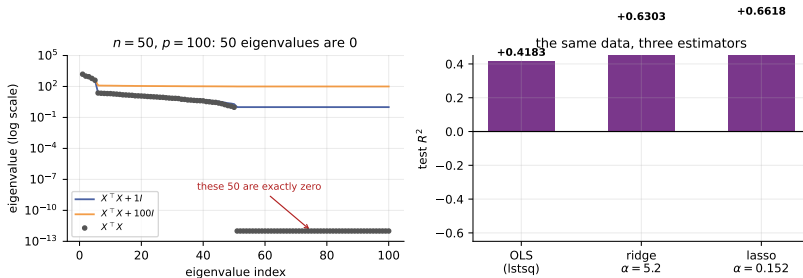
Same shape, but the 100 columns are driven by five latent factors and **all** 100 coefficients are non-zero. One draw, then twenty.

```
n = 50, p = 100, columns driven by 5 latent factors (rho = 0.9),
mean |correlation| between distinct columns: 0.3370
  OLS (lstsq min-norm)      train R2 +1.0000  test R2 +0.4183  nonzero 100
  Ridge, CV alpha 5.2      train R2 +0.9469  test R2 +0.6303  nonzero 100
  Lasso, CV alpha 0.152    train R2 +0.8298  test R2 +0.6618  nonzero 14
one draw settles nothing at n = 50. repeating over 20 draws,
seeds SEED..SEED+19:
  mean test R2   OLS +0.4446  ridge +0.6532  lasso +0.6443
  worst OLS draw -0.4915; OLS is worse than predicting the mean in 1 of 20
  ridge beats OLS in 19 of 20 draws, lasso in 16 of 20
```

One draw of a $p > n$ problem is not an experiment

On this draw the lasso wins. Over twenty draws ridge beats OLS 19 times — a large, reliable gain — and beats lasso only 16 times, and by 0.6532 against 0.6443 on the average. **Sparse truth** → **lasso, by a mile**. **Dense correlated truth** → **ridge, on average and narrowly**. Neither dominates, and that is why both are on the syllabus.

Drawn: fifty eigenvalues are exactly zero



Left: the eigenvalues of $X^T X$ on a log scale. Fifty of the hundred are 0 — plotted on the floor of the axis because $\log 0$ does not exist. Adding λI raises the whole spectrum by λ . Right: what that is worth on held-out data.

What ridge does, direction by direction (the SVD view)

Write $X = UDV^T$. Then the fitted values are $X\hat{\beta}^{\text{ridge}} = \sum_j u_j \frac{d_j^2}{d_j^2 + \lambda} u_j^T y$.

```
diabetes, standardised: singular values of X
[42.17 25.68 23.09 20.55 17.11 16.32 15.4 13.85 5.88 1.95]
shrinkage factor d_j^2 / (d_j^2 + lambda), per direction
  lambda    d1    d2    d3    d4    d5    d6    d7    d8    d9    d10
      1  0.999 0.998 0.998 0.998 0.997 0.996 0.996 0.995 0.972 0.791
    1000  0.640 0.397 0.348 0.297 0.226 0.210 0.192 0.161 0.033 0.004
effective degrees of freedom df(lambda) = sum d_j^2/(d_j^2+lambda):
  lambda      0  df 10.0000  lambda    1000  df 2.5087
  lambda    100  df 6.5923   lambda   10000  df 0.4042
```

What ridge does, direction by direction (the SVD view)

Write $X = UDV^T$. Then the fitted values are $X\hat{\beta}^{\text{ridge}} = \sum_j u_j \frac{d_j^2}{d_j^2 + \lambda} u_j^T y$.

```
diabetes, standardised: singular values of X
[42.17 25.68 23.09 20.55 17.11 16.32 15.4 13.85 5.88 1.95]
shrinkage factor d_j^2 / (d_j^2 + lambda), per direction
  lambda    d1    d2    d3    d4    d5    d6    d7    d8    d9    d10
    1  0.999 0.998 0.998 0.998 0.997 0.996 0.996 0.995 0.972 0.791
  1000  0.640 0.397 0.348 0.297 0.226 0.210 0.192 0.161 0.033 0.004
effective degrees of freedom df(lambda) = sum d_j^2/(d_j^2+lambda):
  lambda      0  df 10.0000  lambda    1000  df 2.5087
  lambda    100  df 6.5923  lambda   10000  df 0.4042
```

Ridge shrinks the *low-variance* directions hardest

At $\lambda = 1000$ the widest direction keeps 64% of itself and the narrowest keeps 0.4%. Those narrow directions are exactly where the collinearity lives and where the estimates are least reliable. $df(\lambda)$ falls from 10 to 2.5 — the same ten columns, a much smaller model.

Change one exponent

Ridge (L2)

$$\text{RSS} + \lambda \sum_{j=1}^p \beta_j^2$$

Smooth. Differentiable everywhere.
Closed form.

Lasso (L1)

$$\text{RSS} + \lambda \sum_{j=1}^p |\beta_j|$$

Convex, but **not differentiable** at $\beta_j = 0$.
No closed form.

Why the derivative matters so much

$\frac{d}{d\beta} \beta^2 = 2\beta$, which is 0 at $\beta = 0$: a coefficient already at zero feels *no* pull from the L2 penalty, so there is nothing to hold it there.

$\frac{d}{d\beta} |\beta| = \pm 1$, which does not go to zero as $\beta \rightarrow 0$: the L1 penalty pulls with **constant force right up to the origin**, so it can push a coefficient to exactly 0 and hold it.

Ridge shrinks. Lasso shrinks and selects. That one sentence is the examinable content of this section.

Why there is no matrix formula

Setting the gradient to zero needs a gradient. At $\beta_j = 0$ the function $|\beta_j|$ has none; it has a **subdifferential**, the whole interval $[-1, 1]$. The stationarity condition becomes

$$X^T(y - X\beta) = \lambda s, \quad s_j \in \begin{cases} \{\text{sign}(\beta_j)\} & \beta_j \neq 0, \\ [-1, 1] & \beta_j = 0. \end{cases}$$

You cannot invert that

s depends on which coefficients are zero, and which coefficients are zero depends on s . There is no matrix you can pre-multiply by — *unless* $X^T X$ is diagonal, in which case the problem decouples into p one-dimensional problems we can solve exactly. That special case is the next slide, and the general algorithm (**coordinate descent**: fix every coefficient but one, solve that one exactly, repeat) is nothing but that special case applied one coordinate at a time.

By hand #3: the orthonormal case, exactly solvable

Suppose $X^T X = I$. Then $\text{RSS} = \text{const} + \sum_j [(\beta_j - \hat{\beta}_j)^2]$ where $\hat{\beta} = X^T y$ is the OLS answer, so the problem *separates*:

$$\min_{\beta_j} (\beta_j - \hat{\beta}_j)^2 + \lambda \text{pen}(\beta_j) \quad \text{one } j \text{ at a time.}$$

	Solution	Behaviour
ridge	$\frac{\hat{\beta}_j}{1 + \lambda}$	every coefficient scaled by the <i>same factor</i>
lasso	$\text{sign}(\hat{\beta}_j)(\hat{\beta}_j - \frac{\lambda}{2})_+$	every coefficient moved by the <i>same amount</i> , and clipped at 0

The second one has a name: *soft thresholding*

Anything with $|\hat{\beta}_j| \leq \lambda/2$ is set to exactly zero. Not “very small”. **Zero.**

The same four numbers, checked against the library

The OLS coefficients are (3, 1, -0.4, 0.2) on an orthonormal design, and $\lambda = 1$, so the lasso threshold is $\lambda/2 = 0.5$.

```
orthonormal design: X'X = I to 8.9e-16
OLS coefficients      [ 3.   1.  -0.4  0.2]
ridge  beta_j / (1 + lambda)      [ 1.5  0.5 -0.2  0.1]
lasso  sign(beta_j) (|beta_j| - lambda/2)+ [2.5 0.5 0.  0. ]
exact zeros produced by lasso: 2 of 4
Ridge(alpha=1).coef_ =      [ 1.5  0.5 -0.2  0.1]
Lasso(alpha=1/(2n)).coef_ =   [ 2.5  0.5 0.  0. ]
```

The same four numbers, checked against the library

The OLS coefficients are (3, 1, −0.4, 0.2) on an orthonormal design, and $\lambda = 1$, so the lasso threshold is $\lambda/2 = 0.5$.

```
orthonormal design: X'X = I to 8.9e-16
OLS coefficients      [ 3.   1.  -0.4  0.2]
ridge  beta_j / (1 + lambda)      [ 1.5  0.5 -0.2  0.1]
lasso  sign(beta_j) (|beta_j| - lambda/2)+ [ 2.5  0.5  0.   0. ]
exact zeros produced by lasso: 2 of 4
Ridge(alpha=1).coef_ =      [ 1.5  0.5 -0.2  0.1]
Lasso(alpha=1/(2n)).coef_ =  [ 2.5  0.5  0.   0. ]
```

Read the two middle rows against each other

Ridge halved all four: $3 \rightarrow 1.5$, $0.2 \rightarrow 0.1$. Lasso subtracted 0.5 from all four and clipped: $3 \rightarrow 2.5$, and −0.4 and 0.2 became **exactly zero**. **The biggest coefficient is punished hardest by ridge and least, proportionally, by lasso.**

The same four numbers, checked against the library

The OLS coefficients are (3, 1, -0.4, 0.2) on an orthonormal design, and $\lambda = 1$, so the lasso threshold is $\lambda/2 = 0.5$.

```

orthonormal design: X'X = I to 8.9e-16
OLS coefficients      [ 3.   1.  -0.4  0.2]
ridge  beta_j / (1 + lambda)      [ 1.5  0.5 -0.2  0.1]
lasso  sign(beta_j) (|beta_j| - lambda/2)+ [2.5 0.5 0.   0. ]
exact zeros produced by lasso: 2 of 4
Ridge(alpha=1).coef_ =      [ 1.5  0.5 -0.2  0.1]
Lasso(alpha=1/(2n)).coef_ =   [ 2.5  0.5 0.   0. ]

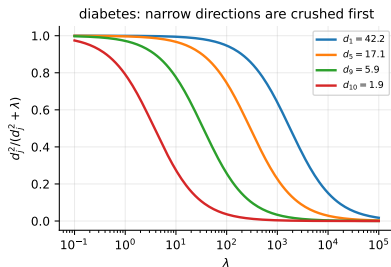
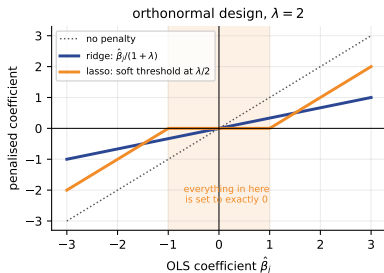
```

Read the two middle rows against each other

Ridge halved all four: $3 \rightarrow 1.5$, $0.2 \rightarrow 0.1$. Lasso subtracted 0.5 from all four and clipped: $3 \rightarrow 2.5$, and -0.4 and 0.2 became **exactly zero**. **The biggest coefficient is punished hardest by ridge and least, proportionally, by lasso.**

Units warning: $\text{Ridge}(\alpha=a)$ minimises $\|y - X\beta\|^2 + a\|\beta\|^2$, so $a = \lambda$; $\text{Lasso}(\alpha=a)$ minimises $\frac{1}{2n}\|y - X\beta\|^2 + a\|\beta\|_1$, so $\lambda = 2na$. Here $n = 200$, hence $a = 1/400$.

Drawn: two shrinkage rules



Left: ridge is a straight line through the origin with slope $1/(1 + \lambda)$ — it touches zero only at zero. Lasso is a straight line *shifted*, with a flat dead zone of width λ in the middle. Right: on real data, the direction with the smallest singular value is crushed first.

No formula, so: twenty lines of coordinate descent

```
def lasso_cd(X, y, alpha, iters=500): #  $(1/2n)||y-Xb||^2 + \alpha||b||_1$ 
    n, p = X.shape; b = np.zeros(p); r = y - X @ b
    norms = (X ** 2).sum(axis=0)
    for _ in range(iters):
        for j in range(p):
            r += X[:, j] * b[j] # take column j out
            rho = X[:, j] @ r / n
            b[j] = soft_threshold(rho, alpha) / (norms[j] / n)
            r -= X[:, j] * b[j] # put it back
    return b
```

```
alpha 2.0 my 20-line coordinate descent vs sklearn: max |difference| 9.29e-12
mine    [ 0.      -7.5682  24.6228  13.1778  -2.7169  0.      -10.0536  0.    23.1479  1.6904]
sklearn [ 0.      -7.5682  24.6228  13.1778  -2.7169  0.      -10.0536  0.    23.1479  1.6904]
exact zeros: mine 3, sklearn 3
```

No formula, so: twenty lines of coordinate descent

```
def lasso_cd(X, y, alpha, iters=500): #  $(1/2n)||y-Xb||^2 + \alpha||b||_1$ 
    n, p = X.shape; b = np.zeros(p); r = y - X @ b
    norms = (X ** 2).sum(axis=0)
    for _ in range(iters):
        for j in range(p):
            r += X[:, j] * b[j] # take column j out
            rho = X[:, j] @ r / n
            b[j] = soft_threshold(rho, alpha) / (norms[j] / n)
            r -= X[:, j] * b[j] # put it back
    return b
```

```
alpha 2.0 my 20-line coordinate descent vs sklearn: max |difference| 9.29e-12
mine    [ 0.      -7.5682  24.6228  13.1778 -2.7169  0.      -10.0536  0.    23.1479  1.6904]
sklearn [ 0.      -7.5682  24.6228  13.1778 -2.7169  0.      -10.0536  0.    23.1479  1.6904]
exact zeros: mine 3, sklearn 3
```

Agreement to 10^{-11} on all ten coefficients. **The zeros are exact because the update assigns zero; it does not approach it.** That is the entire difference from ridge, expressed as one line of code.

The geometry: a diamond has corners, a disc does not

The constrained picture

The RSS contours are ellipses centred on $\hat{\beta}^{\text{OLS}}$. The solution is where the smallest ellipse first *touches* the constraint region.

The two regions in two dimensions

$\|\beta\|_1 \leq t$ is a **diamond**, whose four corners sit *on the axes*.

$\|\beta\|_2 \leq s$ is a **disc**, which is smooth everywhere and has no distinguished points at all.

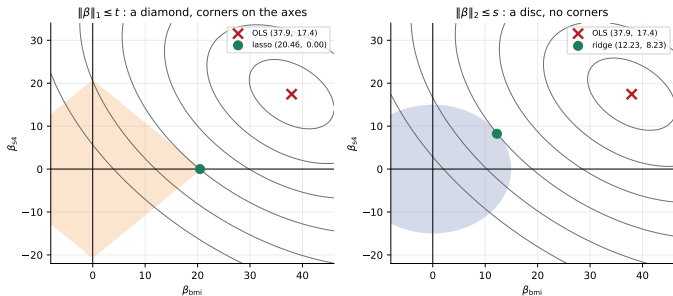
Why corners produce zeros

A corner is where the boundary has a *kink*. An ellipse arriving from a generic direction will meet a kink before it meets any smooth part — and a corner of the diamond has $\beta_j = 0$ for every j but one.

A disc has no kink, so the touching point is generically in the interior of a quadrant: **all coefficients non-zero**.

In p dimensions the L1 ball has corners, edges and faces of every dimension from 0 to $p - 1$ — a whole hierarchy of ways to be sparse.

Drawn: the same budget, two answers



Two standardised diabetes columns. OLS is at (37.9, 17.4). Constrain to $\|\beta\|_1 \leq 20.46$ and the answer is the corner (20.46, 0.00); give ridge *the same L1 budget* and it answers (12.23, 8.23).

Same budget, and one of them deletes a predictor.

Predict first #3

Standardised diabetes, ten features. Sweep α from tiny to large under both penalties.

At $\alpha = 100$, how many ridge coefficients are exactly zero? And how many lasso coefficients, at the α that gives the same amount of shrinkage?

Predict first #3

Standardised diabetes, ten features. Sweep α from tiny to large under both penalties.

At $\alpha = 100$, how many ridge coefficients are exactly zero? And how many lasso coefficients, at the α that gives the same amount of shrinkage?

alpha	ridge nonzero	min coef	lasso nonzero	kept
ridge a=0.01	lasso a=0.000	10	0.4756	10 age, sex, bmi, bp, s1, s2, s3, s4, s5, s6
ridge a=10.00	lasso a=0.100	10	0.2579	9 age, sex, bmi, bp, s1, s2, s4, s5, s6
ridge a=50.00	lasso a=0.500	10	0.1070	8 sex, bmi, bp, s1, s3, s4, s5, s6
ridge a=100.00	lasso a=1.000	10	0.4361	7 sex, bmi, bp, s1, s3, s5, s6

ridge never returns an exact zero: smallest |coef| over the whole grid
1.455e-03 at alpha up to 1e6 -- small, never 0

Predict first #3

Standardised diabetes, ten features. Sweep α from tiny to large under both penalties.

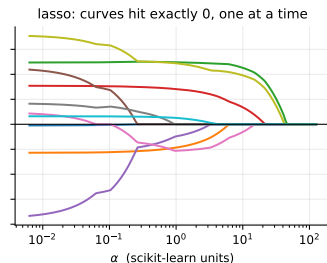
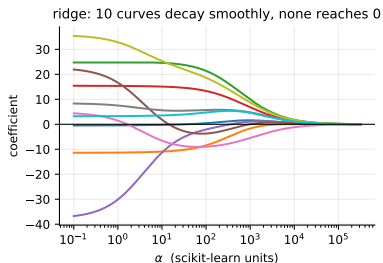
At $\alpha = 100$, how many ridge coefficients are exactly zero? And how many lasso coefficients, at the α that gives the same amount of shrinkage?

```
alpha      ridge nonzero  min|coef|  lasso nonzero  kept
ridge a=0.01  lasso a=0.000    10      0.4756      10  age, sex, bmi, bp, s1, s2, s3, s4,
s5, s6
ridge a=10.00  lasso a=0.100    10      0.2579       9  age, sex, bmi, bp, s1, s2, s4, s5,
s6
ridge a=50.00  lasso a=0.500    10      0.1070       8  sex, bmi, bp, s1, s3, s4, s5, s6
ridge a=100.00 lasso a=1.000    10      0.4361       7  sex, bmi, bp, s1, s3, s5, s6
ridge never returns an exact zero: smallest |coef| over the whole grid
1.455e-03 at alpha up to 1e6 -- small, never 0
```

Ridge: ten, ten, ten, ten. For ever.

Even at $\alpha = 10^6$, where the coefficients are of order 10^{-3} , none of them is zero. **Ridge gives you a smaller model; lasso gives you a shorter one.** If the deliverable is “which five measurements do we actually need”, only one of those helps.

Drawn: the figure this lecture is for



— age — bmi — s1 — s3 — s5
— sex — bp — s2 — s4 — s6

Ten coefficients, one colour each, as α sweeps six orders of magnitude. **Left: ten curves glide towards the axis and never arrive. Right: they land on it, one at a time, and stay.**

The order of deletion is the ranking

as alpha RISES, lasso coefficients hit exactly zero in this order:

1. age	leaves at alpha = 0.2473	6. sex	leaves at alpha = 6.3339
2. s2	leaves at alpha = 0.2641	7. s3	leaves at alpha = 15.3382
3. s4	leaves at alpha = 0.9790	8. bmi	never
4. s1	leaves at alpha = 3.2897	9. bp	never
5. s6	leaves at alpha = 4.2753	10. s5	never
alpha	5.00	nonzero	5 R2 on the training rows +0.4892
alpha	0.05	nonzero	10 R2 on the training rows +0.5176

The order of deletion is the ranking

as alpha RISES, lasso coefficients hit exactly zero in this order:

1. age	leaves at alpha = 0.2473	6. sex	leaves at alpha = 6.3339
2. s2	leaves at alpha = 0.2641	7. s3	leaves at alpha = 15.3382
3. s4	leaves at alpha = 0.9790	8. bmi	never
4. s1	leaves at alpha = 3.2897	9. bp	never
5. s6	leaves at alpha = 4.2753	10. s5	never
alpha	5.00	nonzero	5 R2 on the training rows +0.4892
alpha	0.05	nonzero	10 R2 on the training rows +0.5176

Half the features and 94.5% of the training R^2 . The last three standing — body mass index, blood pressure and s5 — are the three a clinician would have named.

The order of deletion is the ranking

as alpha RISES, lasso coefficients hit exactly zero in this order:

1. age	leaves at alpha = 0.2473	6. sex	leaves at alpha = 6.3339
2. s2	leaves at alpha = 0.2641	7. s3	leaves at alpha = 15.3382
3. s4	leaves at alpha = 0.9790	8. bmi	never
4. s1	leaves at alpha = 3.2897	9. bp	never
5. s6	leaves at alpha = 4.2753	10. s5	never
alpha	5.00	nonzero	5 R2 on the training rows +0.4892
alpha	0.05	nonzero	10 R2 on the training rows +0.5176

Half the features and 94.5% of the training R^2 . The last three standing — body mass index, blood pressure and s5 — are the three a clinician would have named.

the count is not perfectly monotone, and that is real, not a bug:

alpha	0.15	nonzero	10	coefficient on s3	-2.13916
alpha	0.10	nonzero	9	coefficient on s3	-0.00000
alpha	0.05	nonzero	10	coefficient on s3	+0.95082

Where the penalty budget actually goes

$\lambda \sum_j \beta_j^2$ adds up p numbers as if they were commensurable. **They are not.** Multiply column j by c : under OLS β_j becomes β_j/c and every prediction is unchanged; under ridge its penalty contribution becomes $\lambda \beta_j^2/c^2$. So a column recorded in *large* numbers is effectively unregularised, and one recorded in *small* numbers gets crushed.

`load_diabetes(scaled=False)`, ridge with $\alpha = 100$.

name	column sd	ridge coef	lambda*coef^2	share of penalty
age	13.295	-0.1608	2.59	0.0002
sex	0.500	-7.2829	5304.01	0.3758
bmi	4.480	6.7139	4507.62	0.3194
bp	13.846	0.9030	81.55	0.0058
s1	34.433	1.3138	172.62	0.0122
s5	0.539	5.7095	3259.84	0.2310
s6	12.321	0.1567	2.46	0.0002

Where the penalty budget actually goes

$\lambda \sum_j \beta_j^2$ adds up p numbers as if they were commensurable. **They are not.** Multiply column j by c : under OLS β_j becomes β_j/c and every prediction is unchanged; under ridge its penalty contribution becomes $\lambda \beta_j^2/c^2$. So a column recorded in *large* numbers is effectively unregularised, and one recorded in *small* numbers gets crushed.

`load_diabetes(scaled=False)`, ridge with $\alpha = 100$.

name	column sd	ridge coef	lambda*coef^2	share of penalty
age	13.295	-0.1608	2.59	0.0002
sex	0.500	-7.2829	5304.01	0.3758
bmi	4.480	6.7139	4507.62	0.3194
bp	13.846	0.9030	81.55	0.0058
s1	34.433	1.3138	172.62	0.0122
s5	0.539	5.7095	3259.84	0.2310
s6	12.321	0.1567	2.46	0.0002

Three narrow columns pay 93% of the penalty

sex (sd 0.50), bmi (4.48) and s5 (0.54) together take $0.3758 + 0.3194 + 0.2310 = \mathbf{0.9262}$ of the budget. age and s6, whose standard deviations are 13, pay 0.0002 each. **The penalty is being spent on the columns with the smallest numbers, not the ones with the least evidence.**

Predict first #4

Take `bmi` and divide it by 100 — report it as a fraction rather than in kg per square metre. Nothing else changes; the information is identical.

What happens to the OLS fit? What happens to the ridge fit?

Predict first #4

Take `bmi` and divide it by 100 — report it as a fraction rather than in kg per square metre. Nothing else changes; the information is identical.

What happens to the OLS fit? What happens to the ridge fit?

```
OLS   coef on bmi      6.2458 ->    624.5825   ratio   100.0000
OLS   test R2          +0.392899 -> +0.392899   (the same fit)
ridge coef on bmi      6.7139 ->     2.8018   ratio    0.4173
ridge test R2          +0.358505 -> +0.309806   (a DIFFERENT model)
bmi's share of the ridge penalty 0.3194 -> 0.0410
max |prediction difference|, OLS  8.697e-12
max |prediction difference|, ridge 71.0177
```

Predict first #4

Take `bmi` and divide it by 100 — report it as a fraction rather than in kg per square metre. Nothing else changes; the information is identical.

What happens to the OLS fit? What happens to the ridge fit?

```

OLS   coef on bmi      6.2458 ->    624.5825   ratio   100.0000
OLS   test R2          +0.392899 -> +0.392899   (the same fit)
ridge coef on bmi      6.7139 ->     2.8018   ratio    0.4173
ridge test R2          +0.358505 -> +0.309806   (a DIFFERENT model)
bmi's share of the ridge penalty 0.3194 -> 0.0410
max |prediction difference|, OLS  8.697e-12
max |prediction difference|, ridge 71.0177
  
```

8.7×10^{-12} against 71.0

OLS is **equivariant**: the coefficient scales by exactly 100 and the predictions do not move by a picogram. Ridge is not: the coefficient moves by a factor of 0.42, and predictions on the same patients shift by up to 71 units of a target that ranges from 25 to 346. **A change of unit changed the model.**

And under lasso, a change of unit deletes a predictor

the same unit change under lasso, which also selects:

```
alpha 1  kg/m^2 units: kept 9  age sex bmi bp s1 s2 s3 s5 s6
```

```
alpha 1  bmi/100      : kept 7  age sex bp s2 s3 s5 s6
```

And under lasso, a change of unit deletes a predictor

the same unit change under lasso, which also selects:

```
alpha 1  kg/m^2 units: kept 9  age sex bmi bp s1 s2 s3 s5 s6
```

```
alpha 1  bmi/100      : kept 7  age sex bp s2 s3 s5 s6
```

bmi is gone. The single most predictive variable in the dataset was removed from the model, not because the evidence changed but because somebody chose to divide it by a hundred. s1 went with it.

And under lasso, a change of unit deletes a predictor

the same unit change under lasso, which also selects:

alpha 1 kg/m² units: kept 9 age sex bmi bp s1 s2 s3 s5 s6

alpha 1 bmi/100 : kept 7 age sex bp s2 s3 s5 s6

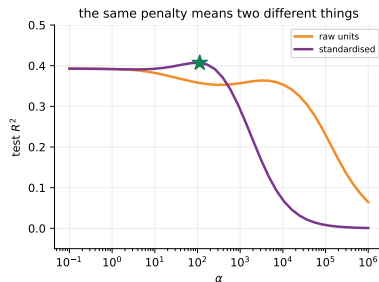
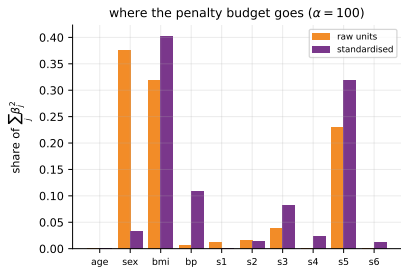
bmi is gone. The single most predictive variable in the dataset was removed from the model, not because the evidence changed but because somebody chose to divide it by a hundred. s1 went with it.

ridge across a grid of alpha, raw units against standardised:

alpha	raw test R2	standardised test R2
1	+0.3919	+0.3916
100	+0.3585	+0.4073
10000	+0.3539	+0.0699

Standardised, the best available α reaches 0.4073; on raw units the curve never gets above 0.3919, and it gets there by turning the penalty *off*.

Drawn: two panels, one rule



Left: the share of $\sum_j \beta_j^2$ each column consumes, raw against standardised. Right: test R^2 against α . **The orange curve is not a worse version of the purple one — it is a different experiment, in which α means something else.**

The intercept is not penalised, and here is the proof

Shift every target up by 500 and refit.

```
ridge with alpha = 100, then the same fit with y shifted up by 500
intercept          152.1197 ->      652.1197   difference 500.0000
max |coefficient difference|  1.288e-14
mean of y = 152.1197, intercept of the unshifted fit = 152.1197
```

The intercept is not penalised, and here is the proof

Shift every target up by 500 and refit.

```
ridge with alpha = 100, then the same fit with y shifted up by 500
```

```
intercept      152.1197 ->      652.1197   difference 500.0000
```

```
max |coefficient difference|  1.288e-14
```

```
mean of y = 152.1197, intercept of the unshifted fit = 152.1197
```

```
what happens if you DO penalise it -- fit_intercept=False on uncentred y:
```

```
alpha          1   with intercept R2 +0.3916   without -3.8362
```

```
alpha         100  with intercept R2 +0.4073   without -3.8990
```

```
Ridge(alpha=1e12): intercept 152.1197, max|coef| 1.512e-08, prediction 152.1197
```

The intercept is not penalised, and here is the proof

Shift every target up by 500 and refit.

```
ridge with alpha = 100, then the same fit with y shifted up by 500
intercept      152.1197 ->      652.1197   difference 500.0000
max |coefficient difference|  1.288e-14
mean of y = 152.1197, intercept of the unshifted fit = 152.1197
```

what happens if you DO penalise it -- fit_intercept=False on uncentred y:

```
alpha      1   with intercept R2 +0.3916   without -3.8362
alpha     100  with intercept R2 +0.4073   without -3.8990
Ridge(alpha=1e12): intercept 152.1197, max|coef| 1.512e-08, prediction 152.1197
```

Two things to take away

The intercept absorbed the shift *exactly* (+500.0000) and no coefficient moved by more than 10^{-14} . And it equals $\bar{y}$. Penalise it instead — fit_intercept=False on data whose mean is 152 — and R^2 goes to **-3.84**. As $\lambda \rightarrow \infty$ **ridge must predict a constant, and the right constant is $\bar{y}$.**

The U-curve, the minimum and the 1-SE choice

Hold out a test set and do not look at it. On the training rows take a *logarithmic* grid of λ , run k -fold cross-validation at each, and record the mean validation error **and its standard error**. Then choose — two answers are defensible. **The minimum**: lowest mean CV error.

The one-standard-error rule: the *largest* λ whose mean is still within one standard error of the minimum, because the curve is flat near the bottom and estimated with noise.

```
RIDGE, 5-fold CV on the training rows, 60 alphas from 1e-2 to 1e4
minimum      alpha    22.6951      CV MSE  3050.64      se 219.00
1-SE threshold                CV MSE  3269.64
1-SE choice   alpha    147.7378      CV MSE  3213.05
      alpha    22.6951  test R2 +0.3971  test MSE  3075.84  ||beta||    41.799
      alpha    147.7378  test R2 +0.4049  test MSE  3035.86  ||beta||    32.073
for comparison, OLS (alpha = 0): test R2 +0.3929  test MSE  3097.12
```

The U-curve, the minimum and the 1-SE choice

Hold out a test set and do not look at it. On the training rows take a *logarithmic* grid of λ , run k -fold cross-validation at each, and record the mean validation error **and its standard error**. Then choose — two answers are defensible. **The minimum:** lowest mean CV error.

The one-standard-error rule: the *largest* λ whose mean is still within one standard error of the minimum, because the curve is flat near the bottom and estimated with noise.

RIDGE, 5-fold CV on the training rows, 60 alphas from 1e-2 to 1e4

```

minimum      alpha    22.6951      CV MSE  3050.64      se 219.00
1-SE threshold                CV MSE  3269.64
1-SE choice   alpha    147.7378      CV MSE  3213.05
      alpha    22.6951  test R2 +0.3971  test MSE  3075.84  ||beta||    41.799
      alpha    147.7378  test R2 +0.4049  test MSE  3035.86  ||beta||    32.073
for comparison, OLS (alpha = 0): test R2 +0.3929  test MSE  3097.12

```

LASSO, same folds

```

minimum      alpha    0.9769      test R2 +0.3889  nonzero  8
1-SE choice   alpha    9.6926      test R2 +0.3616  nonzero  4    bmi, bp, s3, s5

```

The U-curve, the minimum and the 1-SE choice

Hold out a test set and do not look at it. On the training rows take a *logarithmic* grid of λ , run k -fold cross-validation at each, and record the mean validation error **and its standard error**. Then choose — two answers are defensible. **The minimum**: lowest mean CV error. **The one-standard-error rule**: the *largest* λ whose mean is still within one standard error of the minimum, because the curve is flat near the bottom and estimated with noise.

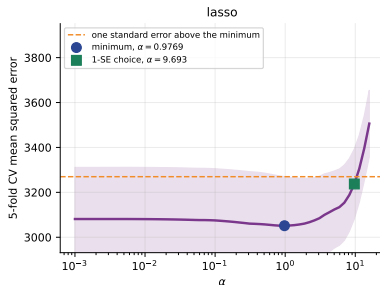
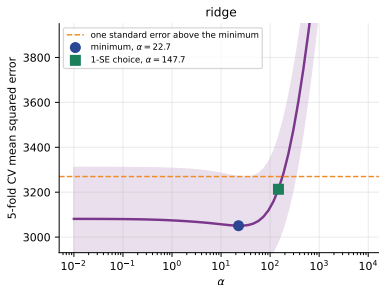
```
RIDGE, 5-fold CV on the training rows, 60 alphas from 1e-2 to 1e4
minimum      alpha    22.6951      CV MSE  3050.64      se 219.00
1-SE threshold                CV MSE  3269.64
1-SE choice   alpha    147.7378      CV MSE  3213.05
      alpha    22.6951  test R2 +0.3971  test MSE  3075.84  ||beta||    41.799
      alpha    147.7378  test R2 +0.4049  test MSE  3035.86  ||beta||    32.073
for comparison, OLS (alpha = 0): test R2 +0.3929  test MSE  3097.12

LASSO, same folds
minimum      alpha    0.9769      test R2 +0.3889  nonzero  8
1-SE choice   alpha    9.6926      test R2 +0.3616  nonzero  4    bmi, bp, s3, s5
```

The 1-SE rule was the better choice for ridge here

$\alpha = 147.7$ beat $\alpha = 22.7$ on the test set (0.4049 against 0.3971) with a coefficient vector 23% shorter — and both beat OLS. For lasso it cost 0.027 of R^2 and bought a model with **four features instead of eight**. That is a trade you may want.

Drawn: read the shape, not just the minimum



The shaded band is $\pm$ one standard error across the five folds. Both curves are **flat for three orders of magnitude** and then climb steeply. The dashed line is the 1-SE threshold; the square is the largest α still under it.

Ridge is biased. On purpose.

Take $X^T X = I$ so the algebra is clean, and write $\hat{\beta}^R = \hat{\beta}^{\text{OLS}} / (1 + \lambda)$. Then

$$\mathbb{E}[\hat{\beta}^R] = \frac{\beta}{1 + \lambda}, \quad \text{bias} = -\frac{\lambda}{1 + \lambda}\beta, \quad \text{Var}(\hat{\beta}^R) = \frac{\sigma^2}{(1 + \lambda)^2}I.$$

So the total squared error is

$$\text{MSE}(\lambda) = \underbrace{\frac{\lambda^2 \|\beta\|^2}{(1 + \lambda)^2}}_{\text{bias}^2, \uparrow} + \underbrace{\frac{\sigma^2 p}{(1 + \lambda)^2}}_{\text{variance}, \downarrow} = \frac{\lambda^2 \|\beta\|^2 + \sigma^2 p}{(1 + \lambda)^2}.$$

Differentiate and something remarkable falls out

$$\text{MSE}'(\lambda) = \frac{2(\lambda \|\beta\|^2 - \sigma^2 p)}{(1 + \lambda)^3}, \text{ so the minimum is at } \boxed{\lambda^* = \sigma^2 p / \|\beta\|^2} \text{ — and}$$

$\text{MSE}'(0) = -2\sigma^2 p < 0$. $\lambda = 0$ is never optimal. There is always a positive λ that beats least squares.

Measured, on 400 training sets

$p = 25$, $n = 40$, $\sigma = 2$. Redraw the training data 400 times, fit ridge at each λ , and decompose the error on 500 fixed test points.

lambda	bias ²	variance	noise	expected test MSE
0.010	0.0226	6.9702	4.00	10.9928
1.805	0.1585	4.3535	4.00	8.5119
4.781	0.6766	3.1703	4.00	7.8469
170.125	12.1011	0.3214	4.00	16.4225
3162.278	17.2243	0.0022	4.00	21.2265

variance removed 3.7998, bias added 0.6540, net gain 3.1459

the net gain is 45.0% of the reducible error ($10.9928 - 4.00 = 6.9928$)

check: $\text{bias}^2 + \text{variance}$ reproduces the measured squared error to $7.11\text{e-}15$

theory says the best lambda is $\sigma^2 p / ||\beta||^2 = 5.4298$

measured optimum 4.781

Measured, on 400 training sets

$p = 25$, $n = 40$, $\sigma = 2$. Redraw the training data 400 times, fit ridge at each λ , and decompose the error on 500 fixed test points.

lambda	bias^2	variance	noise	expected test MSE
0.010	0.0226	6.9702	4.00	10.9928
1.805	0.1585	4.3535	4.00	8.5119
4.781	0.6766	3.1703	4.00	7.8469
170.125	12.1011	0.3214	4.00	16.4225
3162.278	17.2243	0.0022	4.00	21.2265

variance removed 3.7998, bias added 0.6540, net gain 3.1459

the net gain is 45.0% of the reducible error ($10.9928 - 4.00 = 6.9928$)

check: bias^2 + variance reproduces the measured squared error to $7.11\text{e-}15$

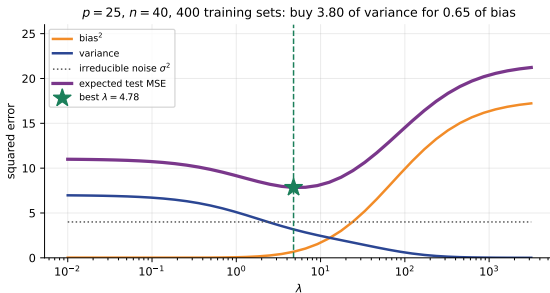
theory says the best lambda is $\sigma^2 p / ||\beta||^2 = 5.4298$

measured optimum 4.781

0.65 of bias bought 3.80 of variance

A net saving of **3.15**, which is 45% of the reducible error. And the closed form landed in the right place without being exact: $\lambda^* = 5.43$ against a measured 4.78, 14% high, because $X^T X$ here is Wishart and not I . **Regularization is not a trick; it is an accounting identity you can compute in advance.**

Drawn: the two curves that cross



Bias² climbs from nothing, variance falls to nothing, and their sum has a minimum strictly to the right of zero. The dotted line is $\sigma^2 = 4$, the part no model can remove. **The right-hand end of the purple curve is the constant model; the left-hand end is least squares.**

Why a third penalty exists

$$\text{RSS} + \lambda \left[\rho \sum_j |\beta_j| + (1 - \rho) \sum_j \beta_j^2 \right], \quad \rho \in [0, 1].$$

In scikit-learn: `ElasticNet(alpha= $\lambda$ , l1_ratio= $\rho$ )`. $\rho = 1$ is lasso, $\rho = 0$ is ridge.

The problem it fixes

Given a group of strongly correlated predictors that all carry the signal, **lasso picks one and discards the rest** — and which one it picks is close to arbitrary. Resample the data and the answer changes.

Why the L2 term fixes it

$\sum \beta_j^2$ is minimised, for a fixed sum, by spreading the weight *evenly*. So the L2 part wants the group to share; the L1 part still lets the whole group be dropped together. This is called the **grouping effect**.

It also lets you select more than n variables when $\rho > n$, which lasso cannot.

Three near-copies of one signal, and their stability

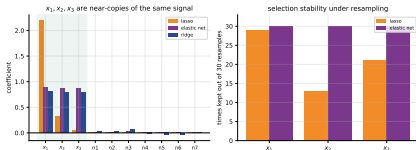
x_1, x_2, x_3 are the same latent variable plus a little noise (pairwise correlation 0.9971); seven pure-noise columns follow.

```
lasso      coefficients on the group [2.1943 0.3251 0.0565]
elastic net coefficients on the group [0.8844 0.8606 0.8602]
ridge      coefficients on the group [0.806  0.7954 0.7973]
stability: refit on 30 bootstrap resamples, count how often each is kept
lasso      kept x1 29/30, x2 13/30, x3 21/30
elastic net kept x1 30/30, x2 30/30, x3 30/30
prediction on 400 fresh rows:  lasso +0.9609  elastic net +0.9636  ridge +0.9435
```

Three near-copies of one signal, and their stability

x_1, x_2, x_3 are the same latent variable plus a little noise (pairwise correlation 0.9971); seven pure-noise columns follow.

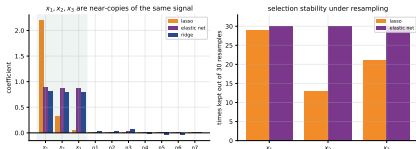
```
lasso      coefficients on the group [2.1943 0.3251 0.0565]
elastic net coefficients on the group [0.8844 0.8606 0.8602]
ridge      coefficients on the group [0.806 0.7954 0.7973]
stability: refit on 30 bootstrap resamples, count how often each is kept
lasso      kept x1 29/30, x2 13/30, x3 21/30
elastic net kept x1 30/30, x2 30/30, x3 30/30
prediction on 400 fresh rows:  lasso +0.9609  elastic net +0.9636  ridge +0.9435
```



Three near-copies of one signal, and their stability

x_1, x_2, x_3 are the same latent variable plus a little noise (pairwise correlation 0.9971); seven pure-noise columns follow.

```
lasso      coefficients on the group [2.1943 0.3251 0.0565]
elastic net coefficients on the group [0.8844 0.8606 0.8602]
ridge      coefficients on the group [0.806  0.7954 0.7973]
stability: refit on 30 bootstrap resamples, count how often each is kept
lasso      kept x1 29/30, x2 13/30, x3 21/30
elastic net kept x1 30/30, x2 30/30, x3 30/30
prediction on 400 fresh rows:  lasso +0.9609 elastic net +0.9636 ridge +0.9435
```



29/30, 13/30, 21/30 against 30/30, 30/30, 30/30

The three columns are indistinguishable, and lasso ranks them anyway: x_1 survives 29 resamples, x_3 survives 21, x_2 only 13. **If you report “the lasso selected x_1 ” you are reporting a coin flip.** Elastic net kept all three every time, and predicted marginally better. **Report a selection with its stability, or do not report it.**

All four on the same data, end to end

```
pipe = make_pipeline(StandardScaler(), RidgeCV(alphas=np.logspace(-2, 4, 60))  
    )  
pipe.fit(X_train, y_train); pipe.score(X_test, y_test)
```

diabetes, 70/30 split, everything inside a Pipeline

model	train R2	test R2	nonzero of 10
OLS	+0.5539	+0.3929	10
Ridge, CV alpha	+0.5505	+0.3989	10
Lasso, CV alpha	+0.5517	+0.3889	8
ElasticNet 0.5	+0.5529	+0.3917	10

All four on the same data, end to end

```
pipe = make_pipeline(StandardScaler(), RidgeCV(alphas=np.logspace(-2, 4, 60))  
    )  
pipe.fit(X_train, y_train); pipe.score(X_test, y_test)
```

diabetes, 70/30 split, everything inside a Pipeline

model	train R2	test R2	nonzero of 10
OLS	+0.5539	+0.3929	10
Ridge, CV alpha	+0.5505	+0.3989	10
Lasso, CV alpha	+0.5517	+0.3889	8
ElasticNet 0.5	+0.5529	+0.3917	10

Report this honestly

On a 442×10 problem with no collinearity crisis, regularization is worth **six thousandths of R^2** . That is the truthful result and you should say so. Its value here is not accuracy: it is that lasso answered the question “which eight?”. **Go back to the $n = 50$, $p = 100$ table for the case where it lifts test R^2 from 0.4183 to 0.6303.**

Summary — twelve lines

1. Regularization means minimising loss + $\lambda \text{pen}(\beta)$. λ is not fitted; it is chosen by cross-validation.
2. Ridge has a closed form, $\hat{\beta} = (X^T X + \lambda I)^{-1} X^T y$, derived in four lines from $\nabla J = 0$.
3. $X^T X + \lambda I$ is positive definite for every $\lambda > 0$, because $v^T (X^T X + \lambda I) v = \|Xv\|^2 + \lambda \|v\|^2$. **Ridge exists where OLS does not.** On a perfectly collinear X , $\det(X^T X) = 0$ and inv raises; add $\lambda = 1$ and the determinant is 71.
4. At $p > n$, LinearRegression does not fail — it silently returns the minimum-norm solution, which is ridge at $\lambda \rightarrow 0$ (agreeing to 5.9×10^{-12}).
5. By hand: $x = (1, 2, 2)$, $y = (2, 3, 5)$ gives $\hat{\beta} = 18/(9 + \lambda)$, so $2.0 \rightarrow 1.8 \rightarrow 1.0$ at $\lambda = 0, 1, 9$; Ridge reproduces all four to four decimals.
6. In the SVD basis, ridge multiplies direction j by $d_j^2 / (d_j^2 + \lambda)$ — the *narrow* directions are crushed first. On diabetes, $\text{df}(\lambda)$ falls $10 \rightarrow 2.51$ at $\lambda = 1000$.
7. Lasso has no closed form, because $|\beta|$ has no derivative at 0. On an orthonormal design it is soft thresholding, and $(3, 1, -0.4, 0.2)$ becomes $(2.5, 0.5, 0, 0)$ — **two exact zeros** — where ridge gives $(1.5, 0.5, -0.2, 0.1)$.
8. Geometrically the L1 ball is a diamond with corners *on the axes*; the L2 ball has none. At the same L1 budget, lasso answered $(20.46, 0)$ and ridge $(12.23, 8.23)$.
9. Scaling is not optional: three narrow raw columns took 92.6% of the ridge penalty, and dividing bmi by 100 moved OLS predictions by 8.7×10^{-12} and ridge predictions by **71.0** — and made lasso **delete** bmi.
10. The intercept is not penalised: shifting y by 500 moved the intercept by exactly 500.0000 and no coefficient by more than 1.3×10^{-14} . Penalise it and $R^2 = -3.84$.
11. Cross-validation gave ridge $\alpha = 22.7$ (test R^2 0.3971) and the 1-SE rule $\alpha = 147.7$ (test R^2 **0.4049**); lasso's 1-SE choice kept **four features of ten**.
12. The trade, measured: $\lambda^* = 4.78$ added 0.65 of bias² and removed 3.80 of variance, a net gain of 3.15 — and the closed form $\sigma^2 p / \|\beta\|^2 = 5.43$ found the same place. $\lambda = 0$ is **never optimal**.

Unit V ends here

Read

notes.pdf — the full derivation, the subdifferential argument for the lasso, the bias–variance algebra, and **10 practice questions with worked solutions**.

Do

Questions 2, 4 and 7. Question 2 is the ridge closed form on a 2×2 matrix by hand; question 7 is soft thresholding; question 4 is the invertibility proof.

What Unit V built

Least squares, maximum likelihood and R^2 ; over-fitting and how to detect it; logistic regression; multiple regression and its pathologies; and now the one dial that controls all of them.

A habit to start now

Never write `Ridge()` or `Lasso()` outside a Pipeline that begins with `StandardScaler`, and never report a coefficient without saying which λ produced it.

Materials: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.; Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.; Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.